

Examples

```
print year(str_to_date("1.1.2024"))           // returns 124
print year(str_to_dateTime("1.1.20 1:00:00")) // returns 120
```

```
print(CalendarYear(sysdate)) // 2024
print(year(sysdate))         // 124
```

See also

[Date and Time Format \[model settings\]](#)

[Model Language \[model settings\]](#)

[CalendarYear \[SimTalk\]](#)

Functions for Communicating with an HTTP Server

Functions for Communicating with an HTTP Server

You can use these functions for sending HTTP requests, for sending data, and for receiving data from the HTTP responses, and for creating and deleting resources on an HTTP server.

- [httpGetRequest \[SimTalk\]](#)
Sends a GET request. Requests data of a resource on an HTTP server.
- [httpPutRequest \[SimTalk\]/httpPutFileRequest \[SimTalk\]](#)
Sends a PUT request with specified data or data contents.
- [httpPostRequest \[SimTalk\]/httpPostFileRequest \[SimTalk\]](#)
Sends a POST request.
- [httpDeleteRequest \[SimTalk\]](#)
Sends a DELETE request.
- [httpHeadRequest \[SimTalk\]](#)
Sends a HEAD request.
- [httpOptionsRequest \[SimTalk\]](#)
Sends an OPTIONS request.

Plant Simulation provides these functions for handling URLs (Unified Resource Locators) by merging URL components or by splitting URL components:

- [httpCreateURL \[SimTalk\]](#)

Creates a correct URL from a JSON structure with URL components.

- [httpSplitURL \[SimTalk\]](#)

Splits the passed URL into its components and saves it as a JSON structure. Also splits existing query or form data to a flat JSON substructure. You can also create a correct URL from this returned JSON structure.

- [httpCreateFormURLEncodedString \[SimTalk\]](#)

Creates the extra component of a URL which is normally inserted after the path of the URL, from a flat JSON structure of name-value-pairs. You can use the result of this function for the payload or the request body.

- [httpSplitFormUrlEncodedString \[SimTalk\]](#)

Creates a flat JSON structure which can also be used in a *httpCreateQuery*, from a query or form string from the extra part of a URL. Data that is converted to a JSON structure is also sent as payload or request body from the server to clients.

Plant Simulation provides these functions for handling payload or request body or response body that is to be sent or received in encoded form. The payload can be used for MQTT and HTTP or for the HTTP request body or response body.

- [encodeStringQuotedPrintable \[SimTalk\]](#)

Encodes the passed string in the Quoted-Printable-encoding where all letters, which are not allowed to be used in the URL without changing its structure, are masked.

- [decodeQuotedPrintableString \[SimTalk\]](#)

Unmasks all letters masked by the Quoted-Printable-encoding to the passed text in its original form.

- [encodeDataBase64 \[SimTalk\]](#) / [encodeBase64FromFile \[SimTalk\]](#)/encodeBase64FromFile

Encodes the entire specified text or the contents of the specified file in the Base64 encoding. The resulting text is no longer readable and cannot be compared with the original data. This way binary data can also be made HTTP transportable.

- [decodeBase64Data \[SimTalk\]](#) / [decodeBase64DataToFile \[SimTalk\]](#)/decodeBase64DataToFile

Decodes the specified data from the Base64 encoding to the original state, transforming binary data from the passed text that was Base64 encoded. The decoded data is either returned to the *Variable* passed as a reference or saved to a file.

- [readBytesFromFile \[SimTalk\]](#)

Reads the specified file and returns the contents as a sequence of byte values in an *array of integer* numbers.

- [writeBytesToFile \[SimTalk\]](#)

Writes the specified *array of integer* numbers as a sequence of byte values to the specified file.

decodeBase64Data [SimTalk] / decodeBase64DataToFile [SimTalk]

Decodes the specified data from the Base64 encoding to the original state, transforming the passed text, that was Base64 encoded, to binary data.

Remarks

The decoded data is either returned as text or as an *array* of *integers*. This depends on the type of the passed variable. If the data contains several bytes with the value 0 (zero) and the passed variable is of data type *string*, an error will occur.

Saves the decoded data to the specified file.

Type

Function

Syntax

```
decodeBase64Data(Base64Data:string, byref Data:string/integer[])  
decodeBase64DataToFile(Base64Data:string, FileName:string)
```

Parameters

You can specify the following parameters:

- The parameter *Base64Data* of data type *string* designates the data to be decoded.
- The parameter *FileName* of data type *string* designates the name of the file to which the decoded data is to be saved.

Example

```
var base64:string := "SGVsbG8gV29ybGQ="
var data:string

decodeBase64Data(base64, data)

print data

// Hello World
```

See also

[encodeDataBase64 \[SimTalk\]](#) / [encodeBase64FromFile \[SimTalk\]](#)

decodeQuotedPrintableString [SimTalk]

Unmasks all letters masked with the Quoted-Printable-encoding in the passed text to its original form.

Type

Function

Syntax

```
decodeQuotedPrintableString(QuotedPrintableText:string) -> string
```

Parameter

The parameter *QuotedPrintableText* of data type *string* designates the data to be decoded.

Return Value

The return value has the data type *string*.

Example

```
print decodeQuotedPrintableString("Hello%20Word")  
// returns Hello World
```

See also

[encodeStringQuotedPrintable \[SimTalk\]](#)

encodeDataBase64 [SimTalk] / encodeBase64FromFile [SimTalk]

Encodes the entire specified data in the Base64 encoding.

Remarks

The resulting text is no longer readable and cannot be compared with the original data. This way binary data can also be made transportable via HTTP or MQTT.

The function *encodeBase64FromFile* encodes the contents of the specified file in the Base64 encoding.

Type

Function

Syntax

```
encodeDataBase64(Data:string/json/integer[]) -> string  
encodeBase64FromFile(FileName:string) -> string
```

Parameters

You can specify the following parameters:

- The parameter *Data* of data type *string/json/integer[]/any*) designates the data to be encoded.
- The parameter *FileName* of data type *string* designates the name of the file containing the data to be encoded.

Return Value

The return value is an *array* of data type *string* with characters within the US-ASCII or 7-bit ASCII character set.

Example

```
var text := "Hello World"  
var base64:string := encodeDataBase64(text)  
print base64  
  
// returns SGVsbG8gV29ybGQ=
```

See also

[decodeBase64Data \[SimTalk\]](#) / [decodeBase64DataToFile \[SimTalk\]](#)

encodeURIComponentPrintable [SimTalk]

Encodes the passed string in the Quoted-Printable-encoding.

Remarks

All letters, which are not allowed to be used in the URL without changing its structure, are masked. Normally these are characters outside of the US-ASCII character set (ASCII 7-bit) and characters which have a function within a URL, such as */*, *:*, *@*, *&*, *=*, *spaces*, etc.

Type

Function

Syntax

```
encodeURIComponentPrintable(Text:string) -> string
```

Parameter

The parameter *Text* of data type *string* designates the text to be encoded.

Return Value

The return value has the data type *string*.

Example

```
print encodeURIComponentPrintable("Hello World")  
// returns Hello%20World
```

See also

[decodeQuotedPrintableString \[SimTalk\]](#)

httpCreateFormURLEncodedString [SimTalk]

Combines the names and values of an HTTP request to a valid text, that can be attached as **extraInfo** after the path.

Remarks

If the names or the values contain illegal letters for a URL, these are masked using the Quoted-Printable-encoding.

Type

Function

Syntax

```
httpCreateFormUrlEncodedString(FormKeyValuePairs:json) -> string
```

Parameter

The parameter *FormKeyValuePairs* of data type *json* designates JSON elements containing the names and values contained within in a flat JSON structure which are to be used as URL query parameters.

A JSON structure might look like this:

```
{
  "name1": "value1",
  "name 2": "value2",
  name3": "value 3",
  ...
}
```

The result for the JSON structure above is a URL query with this format:

```
name1=value1&name%202=value2&name3=value%203&...
```

Return Value

The return value has the data type *string*.

Example

```
var requestHeaders:json
requestHeader["Content-Type"] := "application/x-www-form-urlencoded"
```

```
var data:json
data["name1"] := "value1"
data["name 2"] := "value2",
data["name3"] := "value 3",

var requestBody:string := httpCreateFormUrlEncodedString(data)
var responseHeaders:json
var responseBody:any

// httpPostRequest(url, requestBody, requestHeaders, responseHeaders,
responseBody)

print requestBody

// returns name1=value1&name%202=value2&name3=value%203
```

See also

[httpSplitFormUrlEncodedString \[SimTalk\]](#)

httpCreateURL [SimTalk]

Creates a syntactically correct URL from a JSON structure with URL elements.

Type

Function

Syntax

```
httpCreateURL(URLElements:json) -> string
```

Parameter

The parameter *URLElements* of data type *json* designates the URL elements:

Note

Elements that you do not specify are empty and thus are not part of the URL to be created.

- **SSL:** Is true if the URL describes a transport encryption (https), false without encryption (http).
- **Server:** Is the name of the server, is also called host name.
- **Port:** Is the integer number of the port in the network on the designated server.
- **Extra:** Is the text to be added after the path to the URL or a flat JSON structure. The text can be a reference to a resource section, that starts with **#**, or a URL query text, starting with a **?** (question mark).

The function also accepts a flat JSON structure of URL query parameters which start with a **?** in the URL.

- **UserName:** Is the user name used for the authentication in the URL.
- **Password:** Is the password to the user name used for the authentication in the URL.

The result is a URL with this format:

```
http[s]://[<username>:<password>@]<hostname>[:<port>][/<path>][<extra>]
```

Return Value

The return value has the data type *string*.

Examples

```
var urlData:json
urlData["Server"] := "ServerName"
urlData["Port"] := 80 -- default port
```

```
urlData["Path"] := "/A/B/C"  
urlData["Scheme"] := "http"
```

```
var urlData:json  
urlData["Server"] := "ServerName"  
urlData["Port"] := 8080 -- HTTP port 8080  
urlData["Path"] := "/A/B/C"  
urlData["Scheme"] := "http"
```

httpDeleteRequest [SimTalk]

Sends a DELETE request. It deletes a resource on an HTTP server identified by the specified URL.

Remarks

A DELETE request can send data in the *RequestBody* and receive data from the HTTP server in the *ResponseBody*.

Type

Function

Syntax

```
httpDeleteRequest(URL:string[, RequestHeader:string/json,  
RequestBody:string/json/integer[]/any , byref ResponseHeader:json, byref  
ResponseBody:string/json/integer[]/any]) -> integer
```

Parameters

You can specify the following parameters:

- The parameter *URL* of data type *string* designates the URL. It is used to address a resource on an HTTP server. In addition the URL can also contain authentication information. For more information, consult [httpGetRequest \[SimTalk\]](#).
- The parameter *RequestHeader* of data type *json* or *string* designates a JSON object or a textual representation of a JSON object or a RFC 7230 compliant header string describing the request headers. For more information, consult *httpGetRequest*.
- The parameter *RequestBody* of data type *string/json/integer[]/any* designates the payload that is to be sent.

The *RequestBody* accepts data with these data types:

- For data of data type *string* the data is sent as is, i.e., as text without any conversion.
- For data of data type *json* the passed JSON structure is sent after converting it into a *string*.
- For data of data type *array* with *integer* values each number is treated like a byte of the binary data and the numbers are thus sent as a sequence of bytes.
- For data of data type *array* with values of any data type, *Plant Simulation* converts the values of the array into text. The transmitted data are compatible to JSON arrays and will then be automatically transmitted with the content type **application/json**.

Compare these examples:

```
var requestBody1:real[] := [ 1.1, 2.2, 3.3, 4.4, 5.5 ]
var status:integer      := httpPostRequest(url, requestBody1)
// post an array of reals, the Content-Type is application/jsonJSON

var requestBody2:real[] := [ "A", "B", "C", "D", "E" ]
status                  := httpPostRequest(url, requestBody2)
// post an array of strings, the Content-Type is application/json

var requestBody3:real[] := [ { "A": 1 }, { "B": 2 }, { "C": 3 }, { "D":
4 }, { "E": 5 } ]
status                  := httpPostRequest(url, requestBody3)
// post an array of JSON objects, the Content-Type is application/json

var requestBody4:any[] := [ 1, 2.2, 3.3km, "4th element", { "Name":,
"5th element" }, false ]
status                  := httpPostRequest(url, requestBody4)
// post the mixed array of the Content-Type application/json:
// [ 1, 2.2, 3300, "4th element", { "Name":, "5th element" }, false ]
```

If you do not want to transmit an array with the content type **application/json**, you have to specify the content type in the request header.

Compare this example:

```
var requestBody:real[] := [ 1.1, 2.2, 3.3, 4.4, 5.5 ]
var requestHeader:string := "Content-Type: text/plain"
var status:integer      := httpPostRequest(url, requestBody,
requestHeader)
// post an array of reals, the Content-Type is text/plain

var jsRequestHeader:json := { "Content-Type": "text/plain" }
status                    := httpPostRequest(url, requestBody,
jsRequestHeader)
// post an array of reals, the Content-Type is text/plain
```

Note

Physical values, such as length, weight, etc, are converted to their base unit during conversion to text and lose their unit, similar to using the function print.

Compare this example:

```
var len:length := 3.3km
print len // -> 3300
```

```
var status:integer := httpPostRequest(url, len)
// posts the length given in km as m: 3.3km -> 3300
```

Note

Arrays of data type *integer* still have the specific task to map a sequence of bytes and thus binary data, and are not converted to text.

Compare this example:

```
var bytes:integer[] := [ 72, 101, 108, 108, 111, 32, 87, 111, 114,
108, 100, 33 ] // "Hello World!" in bytes

var bytesAsString:string := bytes_to_str("UTF-8", bytes)
// converts the byte array to a string

print bytesAsString // -> Hello World!
var status:integer := httpPostRequest(url, bytes)
// posts the byte array as binary data with the Content-Type
application/octet-stream
```

- If the data of data type *any*
 - ◊ designates a *list object* with the first column of data type *integer* containing corresponding numbers, those numbers are used like an *array* of *integer* values. Those numbers are treated as binary data and sent as a sequence of bytes.
 - ◊ designates integer numbers, floating point numbers, physical values, and other data, these are sent after being converted to text, just like using the function `to_str` does. Physical values are sent with their units.

If no content type is passed in the parameter *RequestHeader* and the data passed as *RequestBody* is of data type *string* or *json*, *Plant Simulation* automatically adds the respective content type **text/plain** or **application/json** to the request-header.

If an *array* of *integer* values is passed as a sequence of bytes and if no specific content type for the binary data is provided, the content type is automatically set to **application/octet-stream**.

If the request header contains the content type **application/x-www-form-urlencoded** and if the request body is of data type *json*, it will automatically be converted to the corresponding text (compare [httpCreateFormURLEncodedString](#)) and sent as payload.

- The parameter *ResponseHeader* designates a variable of data type *json* to which the response header is going to be written. The scheme of the returned response header matches the format of the request header. For more information, consult *httpGetRequest*.
- The parameter *ResponseBody* of data type *string/json/integer[]/any* designates the payload or response body, which the HTTP server returned for a query of a resource, and which are written to the specified variables. For more information, consult *httpGetRequest*.

Default Timeouts

The default timeouts are:

- **10** seconds for **connecting** to the resolved host
- **5** seconds for **sending** a request
- **5** seconds for **receiving** a request response

Return Value

The return value has the data type *integer*.

It represents the HTTP status code, for example:

Status code	means
200	OK. The request was successful.
401	Unauthorized. Authentication is required and has failed or has not been provided yet.
404	Not found. The requested resource was not found.
501	Not implemented. The server does not recognize the requested method or it cannot fulfill the request.

You can find a complete list of HTTP status codes on the web, for example on Wikipedia.

See also

[httpSetTimeouts \[SimTalk\]](#)

[httpGetTimeouts \[SimTalk\]](#)

httpGetRequest [SimTalk]

Sends a GET request and requests data of a resource on an HTTP server.

Type

Function

Syntax

```
httpGetRequest(URL:string, byref ResponseBody:string/json/integer[]/any[,
RequestHeader:string/json, byref ResponseHeader:json]) -> integer
```

Parameters

You can specify the following parameters:

- The parameter *URL* of data type *string* designates the URL (Unified Resource Locator). It is used to address a resource on an HTTP server. In addition the URL can also contain authentication information.

```
http[s]://[<userName>:<password>@]<hostName>[:<port>][/<path>]
[<extraInfo>]
```

The format of a URL looks like this:

- **http[s]://** The scheme of an URL http:// or https:// for an encrypted communication. We do not support other schemata.
- **hostname:** Designates the name of the server, for example www.siemens.com.
- **port:** Designates the port number which describes a part of the network address of the HTTP server. Each network protocol is assigned a standard port, such as 80. If the port for accessing the HTTP server differs, you have to separate it with a colon from the name of the server.
- **path:** Designates the path to the resource that is accessed to create, query, change, or delete it. Separate the components of a path with a forward slash (/).
- **extraInfo:** This part of the URL can reference sections of the data of the resource or contain named parameters in the style of **form-url-coded data**:

To access a part of a HTML page directly, use this scheme:

```
<hostName>/<path>#section
```

Specify parameters like this:

```
<hostName>/<path>?name1=value1&name2=value2&...
```

Parameter values may not contain any characters used for the HTTP protocol or for the URLs.

These characters must be escaped/masked by a preceding percentage sign followed by the hexadecimal representation of their Unicode code points.

Character	URL compliant	Meaning
Empty space ()	%20	forbidden
Forward slash (/)	%2F	in paths only
Ampersand (& sign)	%26	separator of URL parameters
Question mark (?)	%3F	separator of URL parameters of the path
Equal sign (=)	%3D	separator of URL parameters and values
Numerals	%23	only for the reference to sections
Colon (:)	%3A	only in URL schemata between user name and password
At sign (@)	%40	separator of authentication data and name of the server

In addition all characters, which are located outside of the 7-bit-ASCII-character set, have to be masked as described, for example:

Character	URL compliant (UTF-8)
ö	%C3%B6
Ö	%C3%96
ä	%C3%A4
Ä	%C3%84
ü	%C3%BC
Ü	%C3%9C
ß	%C3%9F
€	%E2%82%AC

An example of a form URL parameter might look like this:

```
?w%C3%A4hrungssymbol=%E2%82%AC&w%C3%A4hrungsname=Euro
```

- The parameter *ResponseBody* of data type *string/json/integer[]/any* designates the payload or response body, which the HTTP server returned for a query of a resource, and which is written to the specified variables.

The variables can have these data types:

- Specify a variable of data type *json* to try to use textual data as JSON data. If the data is not usable as JSON data, *Plant Simulation* throws an error.

For a following existing content type of these kinds *Plant Simulation* tries to return the data as JSON structure in any case:

- ◊ **application/json**

- ◊ ***/+json**

If no content type was specified in the response, *Plant Simulation* tests if the data could be used as text, before it tries to use the data as a JSON structure.

- Specify a variable of data type *string* to treat the response data depending on the specified content type. All following content type specifications return the response data as text:

- ◊ **text/***

- ◊ **application/json**

- ◊ ***/+json**

- ◊ ***/+html**

- ◊ ***/+txt**

- ◊ ***/+xml**

If no content type was specified in the response, *Plant Simulation* tests if the data could be used as text. If this is not the case, *Plant Simulation* throws an error.

- Specify a variable as an *array* of data type *integer* to transfer the data byte by byte into this *array*, no matter if you specified a content type or not. This *array* then contains all bytes of the payload of the response of the HTTP server as integer values.

To handle *integer arrays*, representing a sequence of bytes, you can use the following functions:

- ◊ [bytes_to_str \[SimTalk\]](#)

- ◊ [readBytesFromFile \[SimTalk\]](#)

- ◊ [writeBytesToFile \[SimTalk\]](#)

- Specify a variable data type *any* to save the received response data according to their content type to this variable.

- ◊ For the content type **application/json** and ***/+json**, the data is returned as a JSON structure in most cases, except the response body contained a JSON array.

In case a *JSON array* is contained in the response body, an *array* of *boolean*, *integer*, *real*, and *string* values is returned matching the content, if necessary as an *array* of array elements of data type *any*.

- ◊ For the content type **text/***, ***/+html**, ***/+txt**, and ***/+xml**, the data is returned as text.

- ◊ For other content types the data is returned byte by byte in an *array* of *integer values*.

- The parameter *RequestHeader* of data type *json* designates a JSON object or a *string* describing the request headers

If the parameter is of the data type *json*, the following is allowed:

```
{
  "name1": [ "value", ...],
  "name2": [
    "value1": value,
    "value2": value,
    ...
  ],
  "name3": [
    "value1",
    "value2", {
      "value3": value
    }, ...
  ]
}
```

The example above results in:

```
name1: value
name2: value1=value; value2=value;...
name3: value1; value2; value3=value;...
```

If the parameter is of the data type *string*, the following is allowed:

- The textual representation of a JSON object of HTTP headers, for example:
`"{ \"Accept\": [\"application/json\", { \"charset\": \"utf-8\" }] }"`
- A RFC 7230 conform text with http headers, for example:
`"Accept: application/json; charset=utf-8"`

You can use the following letters for names and values:

- Names can only contain letters from the US ASCII character set (7-bit-ASCII).
- Values can only contain letters from the ISO 8859-1 (Latin1) character set.

Plant Simulation checks for these rules before using the specified names and values and throws an error if they are not met. Invalid names or values are not automatically masked by a coding, for example with Quoted-Printable.

We recommend to use the function [encodeURIComponentQuotedPrintable](#).

The *RequestHeader* can be one of the following:

- Text containing a JSON structure, starting with an opening curly brace.
- Ready headers text according to RFC 7230, meeting the syntax from this RFC. Be aware that the text is not checked for validity. Header text that does not start with a letter will be rejected.
- A JSON object mapping the structure and contents of the HTTP headers.

- The parameter *ResponseHeader* designates a variable of data type *json* to which the *response header* is going to be written. The scheme of the returned *response header* matches the format of the *request header*.

As a header can contain several values, separated by a semicolon, all values of a header are returned in a JSON array.

A returned *response header* might look like this:

```
{
  "Status-Line": {
    "HTTP-Version": {
      "Protocol": "HTTP",
      "HTTP-Version": "1.1"
    },
    "Status-Code": 200,
    "Status-Phrase": "OK"
  },
  "Headers": {
    "Connection": ["keep-alive"],
    "Date": ["Wed, 06 Oct 2021 21:27:43 GMT"],
    "Content-Length": ["42"],
    "Content-Type": ["application/json", {
      "charset": "utf-8"
    }],
    "ETag": ["W/\\"2a-a6vQ6tlXYuizn89nQjHv3WKYG7M\\""],
    "Server": ["nginx/1.16.1"],
    "Access-Control-Allow-Origin": ["*"],
    "Access-Control-Allow-Methods": ["GET"],
    "Access-Control-Allow-Headers": ["content type"],
  }
}
```

Default Timeouts

The default timeouts are:

- **10** seconds for **connecting** to the resolved host
- **5** seconds for **sending** a request
- **5** seconds for **receiving** a request response

Return Value

The return value has the data type *integer*.

It represents the HTTP status code, for example:

Status code	means
200	OK. The request was successful.
401	Unauthorized. Authentication is required and has failed or has not been provided yet.
404	Not found. The requested resource was not found.
501	Not implemented. The server does not recognize the requested method or it cannot fulfill the request.

You can find a complete list of HTTP status codes on the web, for example on Wikipedia.

See also

[httpSetTimeouts \[SimTalk\]](#)

[httpGetTimeouts \[SimTalk\]](#)

httpGetTimeouts [SimTalk]

Retrieves the defined timeouts used in the Windows HTTP interface (WinHTTP).

Remarks

All timeout values are specified in milliseconds.

If a value is specified as 0 or -1, the underlying Windows HTTP interface will wait forever and does not use any timeout.

Type

Function

Syntax

```
httpGetTimeouts(byref ResolveTimeout:integer, byref  
ConnectionTimeout:integer, byref SendTimeout:integer, byref  
ReceiveTimeout:integer) -> void
```

Parameters

You can specify the following parameters:

- The parameter *ResolveTimeout* of data type *integer* designates a variable that is passed by reference. It is set to the timeout duration in milliseconds for resolving the host name in an URL.
- The parameter *ConnectTimeout* of data type *integer* designates a variable that is passed by reference. It is set to the timeout duration in milliseconds for connecting to the resolved host.
- The parameter *SendTimeout* of data type *integer* designates a variable that is passed by reference. It is set to the timeout duration in milliseconds for sending a request to a connected host.
- The parameter *ReceiveTimeout* of data type *integer* designates a variable that is passed by reference. It is set to the timeout duration in milliseconds for receiving a request response from a connected host after sending a request.

Return Value

The return value is *void*.

Example

```
var resolve:integer;  
var connect:integer;  
var send:integer;  
var receive:integer;
```

```
httpGetTimeouts(resolve, connect, send, receive)
print to_str("resolve: ", resolve)
print to_str("connect: ", connect)
print to_str("send:    ", send)
print to_str("receive: ", receive)
/* prints, if not changed before by httpSetTimeouts, the defaults (in ms):
resolve: -1
connect: 60000
send:    30000
receive: 30000
*/
```

See also

[httpSetTimeouts \[SimTalk\]](#)

httpHeadRequest [SimTalk]

Sends a HEAD request. A HEAD request resembles a GET request, except of returning data via the response body.

Remarks

This way you can, for example, check what kind of data and how much data would be returned for a GET request. The response body is still part of the syntax though the HTTP server might return error information using the response body.

Type

Function

Syntax

```
httpHeadRequest(URL:string[, RequestHeader:string/json, byref
ResponseHeader:json, byref ResponseBody:string/json/integer[]/any]) ->
integer
```

Parameters

You can specify the following parameters:

- The parameter *URL* of data type *string* designates the URL (Unified Resource Locator). It is used to address a resource on an HTTP server. In addition the URL can also contain authentication information. For more information, consult [httpGetRequest](#).
- The parameter *RequestHeader* of data type *json* or *string* designates a JSON object or a textual representation of a JSON object or a RFC 7230 compliant header string describing the request headers. For more information, consult [httpGetRequest](#).
- The parameter *ResponseHeader* designates a variable of data type *json* to which the response header is going to be written. The scheme of the returned response header matches the format of the request header. For more information, consult [httpGetRequest](#).
- The parameter *ResponseBody* of data type *string/json/integer[]/any* designates the payload or response body, which the HTTP server returned for a query of a resource, and which are written to the specified variables. For more information, consult [httpGetRequest](#).

Default Timeouts

The default timeouts are:

- **10** seconds for **connecting** to the resolved host
- **5** seconds for **sending** a request

- 5 seconds for **receiving** a request response

Return Value

The return value has the data type *integer*.

It represents the HTTP status code, for example:

Status code	means
200	OK. The request was successful.
401	Unauthorized. Authentication is required and has failed or has not been provided yet.
404	Not found. The requested resource was not found.
501	Not implemented. The server does not recognize the requested method or it cannot fulfill the request.

You can find a complete list of HTTP status codes on the web, for example on Wikipedia.

See also

[httpSetTimeouts \[SimTalk\]](#)

[httpGetTimeouts \[SimTalk\]](#)

httpOptionsRequest [SimTalk]

Sends an OPTIONS request.

Remarks

An OPTIONS request queries the access methods (HTTP verbs) and other options for the specified resource.

The response body is still part of the syntax though as the HTTP server might return error information using the response body.

Type

Function

Syntax

```
httpOptionsRequest(URL:string[, RequestHeader:string/json, byref
ResponseHeader:json, byref ResponseBody:string/json/integer[]/any]) ->
integer
```

Parameters

You can specify the following parameters:

- The parameter *URL* of data type *string* designates the URL (Unified Resource Locator). It is used to address a resource on an HTTP server. In addition the URL can also contain authentication information. For more information, consult [httpGetRequest](#).
- The parameter *RequestHeader* of data type *json* or *string* designates a JSON object or a textual representation of a JSON object or a RFC 7230 compliant header string describing the request headers. For more information, consult [httpGetRequest](#).
- The parameter *ResponseHeader* designates a variable of data type *json* to which the response header is going to be written. The scheme of the returned response header matches the format of the request header. For more information, consult [httpGetRequest](#).
- The parameter *ResponseBody* of data type *string/json/integer[]/any* designates the payload or response body, which the HTTP server returned for a query of a resource, and which are written to the specified variables. For more information, consult [httpGetRequest](#).

Default Timeouts

The default timeouts are:

- **10** seconds for **connecting** to the resolved host
- **5** seconds for **sending** a request

- 5 seconds for **receiving** a request response

Return Value

The return value has the data type *integer*.

It represents the HTTP status code, for example:

Status code	means
200	OK. The request was successful.
401	Unauthorized. Authentication is required and has failed or has not been provided yet.
404	Not found. The requested resource was not found.
501	Not implemented. The server does not recognize the requested method or it cannot fulfill the request.

You can find a complete list of HTTP status codes on the web, for example on Wikipedia.

See also

[httpSetTimeouts \[SimTalk\]](#)

[httpGetTimeouts \[SimTalk\]](#)

httpPostFileRequest [SimTalk]

Sends a POST request.

Remarks

As compared to [httpPostRequest](#) an *httpPostFileRequest* uses the contents of the specified file as payload/request body.

Type

Function

Syntax

```
httpPostFileRequest(URL:string, RequestBodyFile:string[,
RequestHeader:string/json, byref ResponseHeader:json, byref
ResponseBody:string/json/integer[ ]/any]) -> integer
```

You can specify the following parameters:

- The parameter *URL* of data type *string* designates the URL (Unified Resource Locator). It is used to address a resource on an HTTP server. In addition the URL can also contain authentication information. For more information, consult [httpGetRequest](#).
- The parameter *RequestBodyFile* of data type *string* designates the file name or the path to a file respectively whose contents is to be sent as payload.
- The parameter *RequestHeader* of data type *json* or *string* designates a JSON object or a textual representation of a JSON object or a RFC 7230 compliant header string describing the request headers. For more information, consult [httpGetRequest](#).
- The parameter *ResponseHeader* designates a variable of data type *json* to which the response header is going to be written. The scheme of the returned response header matches the format of the request header. For more information, consult [httpGetRequest](#).
- The parameter *ResponseBody* of data type *string/json/integer[]/any* designates the payload or response body, which the HTTP server returned for a query of a resource, and which are written to the specified variables. For more information, consult [httpGetRequest](#).

Default Timeouts

The default timeouts are:

- **10** seconds for **connecting** to the resolved host
- **5** seconds for **sending** a request
- **5** seconds for **receiving** a request response

Return Value

The return value has the data type *integer*.

It represents the HTTP status code, for example:

Status code	means
200	OK. The request was successful.
401	Unauthorized. Authentication is required and has failed or has not been provided yet.
404	Not found. The requested resource was not found.
501	Not implemented. The server does not recognize the requested method or it cannot fulfill the request.

You can find a complete list of HTTP status codes on the web, for example on Wikipedia.

See also

[httpSetTimeouts \[SimTalk\]](#)

[httpGetTimeouts \[SimTalk\]](#)

httpPostRequest [SimTalk]

Sends a POST request.

Remarks

httpPostRequest creates or changes a resource on an HTTP server by sending information to this resource as payload or request body.

Normally a client does not receive any data via a response body for a POST request. If an error occurs, a response body is sent back with additional helpful information.

Repeated POST requests with the same data normally will not have the same result.

Type

Function

Syntax

```
httpPostRequest(URL:string, RequestBody:string/json/integer[]/any[,  
RequestHeader:string/json, byref ResponseHeader:json, byref  
ResponseBody:string/json/integer[]/any]) -> integer
```

Parameters

You can specify the following parameters:

- The parameter *URL* of data type *string* designates the URL (Unified Resource Locator). It is used to address a resource on an HTTP server. In addition the URL can also contain authentication information. For more information, consult [httpGetRequest \[SimTalk\]](#).
- The parameter *RequestBody* of data type *string/json/integer[]/any* designates the payload that is to be sent.

The *RequestBody* accepts data with these data types:

- For data of data type *string* the data is sent as is, without any conversion.
- For data of data type *json* the passed JSON structure is sent after converting it into a string.
- For data of data type *array* with *integer* values each number is treated like a byte of the binary data and the numbers are thus sent as a sequence of bytes.
- For data of data type *array* with values of any data type, *Plant Simulation* converts the values of the array into text. The transmitted data are compatible to JSON arrays and will then be automatically transmitted with the content type **application/json**.

Compare these examples:

```
var requestBody1:real[] := [ 1.1, 2.2, 3.3, 4.4, 5.5 ]
var status:integer      := httpPostRequest(url, requestBody1)
// post an array of reals, the Content-Type is application/jsonJSON

var requestBody2:real[] := [ "A", "B", "C", "D", "E" ]
status                  := httpPostRequest(url, requestBody2)
// post an array of strings, the Content-Type is application/json

var requestBody3:real[] := [ { "A": 1 }, { "B": 2 }, { "C": 3 }, { "D":
4 }, { "E": 5 } ]
status                  := httpPostRequest(url, requestBody3)
// post an array of JSON objects, the Content-Type is application/json

var requestBody4:any[] := [ 1, 2.2, 3.3km, "4th element", { "Name":,
"5th element" }, false ]
status                  := httpPostRequest(url, requestBody4)
// post the mixed array of the Content-Type application/json:
// [ 1, 2.2, 3300, "4th element", { "Name":, "5th element" }, false ]
```

If you do not want to transmit an array with the content type **application/json**, you have to specify the content type in the request header.

Compare this example:

```
var requestBody:real[] := [ 1.1, 2.2, 3.3, 4.4, 5.5 ]
var requestHeader:string := "Content-Type: text/plain"
var status:integer      := httpPostRequest(url, requestBody,
requestHeader)
// post an array of reals, the Content-Type is text/plain

var jsRequestHeader:json := { "Content-Type": "text/plain" }
status                  := httpPostRequest(url, requestBody,
jsRequestHeader)
// post an array of reals, the Content-Type is text/plain
```

Note

Physical values, such as length, weight, etc, are converted to their base unit during conversion to text and lose their unit, similar to using the function print.

Compare this example:

```
var len:length := 3.3km
print len // -> 3300
```

```
var status:integer := httpPostRequest(url, len)
// posts the length given in km as m: 3.3km -> 3300
```

Note

Arrays of data type *integer* still have the specific task to map a sequence of bytes and thus binary data, and are not converted to text.

Compare this example:

```
var bytes:integer[] := [ 72, 101, 108, 108, 111, 32, 87, 111, 114,
108, 100, 33 ] // "Hello World!" in bytes

var bytesAsStr:string := bytes_to_str("UTF-8", bytes)
// converts the byte array to a string

print bytesAsStr // -> Hello World!
var status:integer := httpPostRequest(url, bytes)
// posts the byte array as binary data with the Content-Type
application/octet-stream
```

- If the data of data type *any*
 - ◊ designates a *list object* with the first column of data type *integer* containing corresponding numbers, those numbers are used like an *array* of *integer* values. Those numbers are treated as binary data and sent as a sequence of bytes.
 - ◊ designates integer numbers, floating point numbers, physical values, and other data, these are sent after being converted to text, just like using the function [to_str](#). Physical values are sent with their units.

If no content type is passed in the parameter *RequestHeader* and the data passed as *RequestBody* is of data type *string* or *json*, *Plant Simulation* automatically adds the respective content-type **text/plain** or **application/json** to the request-header.

If an *array* of *integer* values is passed as a sequence of bytes and if no specific content type for the binary data is provide, the content type is automatically set to **application/octet-stream**.

If the request header contains the content type **application/x-www-form-urlencoded** and if the request body is of data type *json*, it will automatically be converted to the corresponding text (compare [httpCreateFormURLEncodedString](#)) and sent as payload.

- The parameter *RequestHeader* of data type *json* or *string* designates a JSON object or a textual representation of a JSON object or a RFC 7230 compliant header string describing the request headers. For more information, consult *httpGetRequest*.
- The parameter *ResponseHeader* designates a variable of data type *json* to which the response header is going to be written. The scheme of the returned response header matches the format of the request header. For more information, consult *httpGetRequest*.
- The parameter *ResponseBody* of data type *string/json/integer[]/any* designates the payload or response body, which the HTTP server returned for a query of a resource, and which are written to the specified variables. For more information, consult *httpGetRequest*.

Default Timeouts

The default timeouts are:

- **10** seconds for **connecting** to the resolved host
- **5** seconds for **sending** a request
- **5** seconds for **receiving** a request response

Return Value

It represents the HTTP status code, for example:

It represents the HTTP status code, for example:

Status code	means
200	OK. The request was successful.
401	Unauthorized. Authentication is required and has failed or has not been provided yet.
404	Not found. The requested resource was not found.
501	Not implemented. The server does not recognize the requested method or it cannot fulfill the request.

You can find a complete list of HTTP status codes on the web, for example on Wikipedia.

See also

[httpSetTimeouts \[SimTalk\]](#)

[httpGetTimeouts \[SimTalk\]](#)

httpPutFileRequest [SimTalk]

Sends a PUT request and uses the contents of the specified file as payload (RequestBody).

Type

Function

Syntax

```
httpPutFileRequest(URL:string, RequestBodyFile:string[,
RequestHeader:string/json, byref ResponseHeader:json, byref
ResponseBody:string/json/integer[]/any]) -> integer
```

Parameters

You can specify the following parameters:

- The parameter *URL* of data type *string* designates the URL (Unified Resource Locator). It is used to address a resource on an HTTP server. In addition the URL can also contain authentication information. For more information, consult [httpGetRequest](#).
- The parameter *RequestHeader* of data type *json* or *string* designates a JSON object or a textual representation of a JSON object or a RFC 7230 compliant header string describing the request headers. For more information, consult [httpGetRequest](#).
- The parameter *RequestHeader* of data type *string* or *json* designates a JSON structure describing the request headers. For more information, consult [httpGetRequest](#).
- The parameter *ResponseHeader* designates a variable of data type *json* to which the response header is going to be written. The scheme of the returned response header matches the format of the request header. For more information, consult [httpGetRequest](#).
- The parameter *ResponseBody* of data type *string/json/integer[]/any* designates the payload or response body, which the HTTP server returned for a query of a resource, and which are written to the specified variables. For more information, consult [httpGetRequest](#).

Default Timeouts

The default timeouts are:

- **10** seconds for **connecting** to the resolved host
- **5** seconds for **sending** a request
- **5** seconds for **receiving** a request response

Return Value

The return value has the data type *integer*.

It represents the HTTP status code, for example:

Status code	means
200	OK. The request was successful.
401	Unauthorized. Authentication is required and has failed or has not been provided yet.
404	Not found. The requested resource was not found.
501	Not implemented. The server does not recognize the requested method or it cannot fulfill the request.

You can find a complete list of HTTP status codes on the web, for example on Wikipedia.

See also

[httpSetTimeouts \[SimTalk\]](#)

[httpGetTimeouts \[SimTalk\]](#)

httpPutRequest [SimTalk]

Sends a PUT request.

Remarks

httpPutRequest creates a resource on an HTTP server by sending information to this resource as payload or request body.

Normally a client does not receive any data via a response body for a POST request. If an error occurs, a response body is sent back with additional helpful information.

Repeated requests with the same data as a PUT should always return the same result.

Type

Function

Syntax

```
httpPutRequest(URL:string, RequestBody:string/json/integer[]/any[,
RequestHeader:string/json, byref ResponseHeader:json, byref
ResponseBody:string/json/integer[]/any]) -> integer
```

Parameters

You can specify the following parameters:

- The parameter *URL* of data type *string* designates the URL (Unified Resource Locator). It is used to address a resource on an HTTP server. In addition the URL can also contain authentication information. For more information, consult [httpGetRequest](#).
- The parameter *RequestBody* of data type *string/json/integer[]/any* designates the payload that is to be sent.

The *RequestBody* accepts data with these data types:

- For data of data type *string* the data is sent as is, without any conversion.
- For data of data type *json* the passed JSON structure is sent after converting it into a string.
- For data of data type *array* with *integer* values each number is treated like a byte of the binary data and the numbers are thus sent as a sequence of bytes.
- If the data of data type *any*
 - ◊ designates a list object with the first column of data type *integer* containing corresponding numbers, those numbers are used like an *array* of *integer* values. Those numbers are treated as binary data and sent as a sequence of bytes.

- ♦ designates integer numbers, floating point numbers, physical values, and other data, these are sent after being converted to text, just like using the function [to_str](#). Physical values are sent with their units.

If no content type is passed in the parameter *RequestHeader* and the data passed as *RequestBody* are of data type *string* or *json*, *Plant Simulation* automatically adds the respective content type **text/plain** or **application/json** to the request-header.

If an *array* of *integer* values is passed as a sequence of bytes and if no specific content type for the binary data is provide, the content type is automatically set to **application/octet-stream**.

If the request header contains the content type **application/x-www-form-urlencoded** and if the request body is of data type *json*, it will automatically be converted to the corresponding text (compare [httpCreateFormURLEncodedString](#)) and sent as payload.

- The parameter *RequestHeader* of data type *json* or *string* designates a JSON object or a textual representation of a JSON object or a RFC 7230 compliant header string describing the request headers. For more information, consult [httpGetRequest](#).
- The parameter *ResponseHeader* designates a variable of data type *json* to which the response header is going to be written. The scheme of the returned response header matches the format of the request header. For more information, consult [httpGetRequest](#).
- The parameter *ResponseBody* of data type *string/json/integer[]/any* designates the payload or response body, which the HTTP server returned in the case of sending request data, for example to provide error information or information about the proceeded changes at a resource, and which are written to the specified variables. For more information, consult [httpGetRequest](#).

Default Timeouts

The default timeouts are:

- **10** seconds for **connecting** to the resolved host
- **5** seconds for **sending** a request
- **5** seconds for **receiving** a request response

Return Value

The return value has the data type *integer*.

It represents the HTTP status code, for example:

Status code	means
200	OK. The request was successful.
401	Unauthorized . Authentication is required and has failed or has not been provided yet.

Status code	means
404	Not found. The requested resource was not found.
501	Not implemented. The server does not recognize the requested method or it cannot fulfill the request.

You can find a complete list of HTTP status codes on the web, for example on Wikipedia.

See also

[httpSetTimeouts \[SimTalk\]](#)

[httpGetTimeouts \[SimTalk\]](#)

httpSetTimeouts [SimTalk]

Sets the timeouts used in the Windows HTTP interface (WinHTTP).

Remarks

All timeout values are specified in milliseconds.

If a value is specified as 0 or -1, the timeout is set to infinite waiting.

A negative value, i.e., a value that is not 0 and not -1, will cause an error about an invalid parameter.

Note

Some [Functions for Communicating with an HTTP Server](#) have built-in default timeout values.

Type

Function

Syntax

```
httpSetTimeouts(ResolveTimeout:integer, ConnectionTimeout:integer,  
SendTimeout:integer, ReceiveTimeout:integer) -> void
```

Parameters

You can specify the following parameters:

- The parameter *ResolveTimeout* of data type *integer* designates the duration in milliseconds for the timeout for resolving the host name in an URL.
- The parameter *ConnectTimeout* of data type *integer* designates the duration in milliseconds for the timeout for connecting to the resolved host.
- The parameter *SendTimeout* of data type *integer* designates the duration in milliseconds for the timeout for sending a request to a connected host.
- The parameter *ReceiveTimeout* of data type *integer* designates the duration in milliseconds for the timeout for receiving a request response from a connected host after sending a request.

Return Value

The return value is *void*.

Example

```
/* sets the timeouts to:  
120000ms meaning 120s to resolve an URL  
60000ms meaning 60s to connect to the host  
15000ms meaning 15s to send a request to the host  
15000ms meaning 15s to receive the request response from a host  
*/  
httpSetTimeouts(120000, 60000, 15000, 15000)
```

See also

[httpGetTimeouts \[SimTalk\]](#)

httpSplitFormUrlEncodedString [SimTalk]

Splits the **extraInfo** part of a URL or the text of a response body with the Content-Type **application/x-www-form-urlencoded**.

Remarks

httpSplitFormUrlEncodedString returns a flat JSON structure containing the name-value-pairs recognized in the provided string.

Name and values that contain masked letters encoded with Quoted-Printable-encoded are unmasked.

If you need to extract the **extraInfo** part from a URL and to parse it into name-value-pairs, you can use the function [httpSplitURL](#), since it parses not only the URL, but the **extraInfo** part of the URL as well.

Type

Function

Syntax

```
httpSplitFormUrlEncodedString(FormKeyValueString:string) -> json
```

Parameter

The parameter *FormKeyValuePairs* of data type *string* designates the so-called extra info of an URL that can be found after the path, or is the text from a response body, whose **Content-Type** is **application/x-www-form-urlencoded**.

If the text is part of the URL, it starts with a question mark (?). The text contains name-value-pairs that are separated by an ampersand (&). The names and value themselves are separated by the equal sign (=).

The format of the query parameter specified in the URL looks like this:

```
name1=value1&name%202=value2&name3=value%203&...
```

The question mark is missing in the text received in the response body of a request:

The result is a flat JSON structure containing the recognized names and values of the parameters. Each parameter is a JSON element containing its value:

```
{
  "name1": "value1",
  "name 2": "value2",
  "name3": "value 3",
```



```
} ...
```

Return Value

The return value has the data type *json*.

Example

```
var url:string := "..."  
var respBody:string  
var respHeaders:json  
if httpGetRequest(url, respBody, "", respHeaders) = 200  
    var values:json  
    var contentType:string := respHeaders["Content-Type"]  
  
    if contentType = "application/x-www-form-urlencoded"  
        values := httpSplitFormUrlEncodedString(respBody)  
    elseif contentType = "application/json"  
        values.parse(respBody)  
    else  
        throwRuntimeError("invalid response body content-type")  
    end  
end
```

See also

[httpCreateURL \[SimTalk\]](#)

[httpSplitURL \[SimTalk\]](#)

httpSplitURL [SimTalk]

Splits a URL into its components.

Remarks

You can then change the individual components and put them together again to form a new URL with the function *httpCreateURL*.

Type

Function

Syntax

```
httpSplitURL(URL:string) -> json
```

Parameter

The parameter *URL* of data type *json* designates the URL that you want to split up.

The URL has to have this format:

```
http[s]://[<username>:<password>@]<server>[:<port>][/<path>][<extra>]
```

Return Value

The return value has the data type *json*.

The return value is a flat JSON structure with these elements, provided they were found in the URL:

- **SSL:** Is true if the URL describes a transport encryption (https), false without encryption (http).
- **server:** Name of the server, is also called host name.
- **port:** Integer specification of the port in the network on the designated server.
- **path:** Path to a resource, a directory, or a file on the designated server.
- **extra:** A reference to a resource section, that starts with **#** or a flat JSON structure of a URL query parameter, which start with a **?** in the URL.
- **username:** The user name used for the authentication in the URL.
- **password:** The password to the user name used for the authentication in the URL.
- **scheme:** The scheme of the URL, in this case http or https matching the JSON element SSL.

Note

Plant Simulation does not add elements to the JSON structure that it does not find in the URL and thus are empty.

See also

[httpCreateURL \[SimTalk\]](#)

readBytesFromFile [SimTalk]

Reads the specified file and returns the contents as a sequence of byte values in an *array* of *integer* numbers.

Type

Function

Syntax

```
readBytesFromFile(FileName:string) -> integer[]
```

Parameter

The parameter *FileName* of data type *string* designates the name of the file whose contents is to be converted.

Return Value

The return value is an *array* of data type *integer*.

Example

```
var fileName:string := "..."  
var bytes:integer[] := readBytesFromFile(fileName)  
print bytes  
// prints [72, 101, 108, 108, 111, 32, 87, 111, 114, 108, 100]  
  
var text:string := bytes_to_str("ANSI", bytes)  
print text  
// prints Hello World
```

See also

[writeBytesToFile \[SimTalk\]](#)

writeBytesToFile [SimTalk]

Writes the specified *array* of *integer* numbers as a sequence of byte values to the specified file.

Type

Function

Syntax

```
writeBytesToFile(Bytes:integer[], FileName:string)
```

Parameters

You can specify the following parameters:

- The parameter *Bytes* is an *array* of data type *integer* that designates the sequence of byte values to be written to a file.
- The parameter *FileName* of data type *string* designates the name of the file to contain the specified sequence of bytes.

Example

```

param fileName:string

var bytes:integer[]
var reqHeaders:json
var respHeaders:json

var statusCode:integer := httpGetRequest("http://a.server.com/charts/
resources/current", bytes, reqHeaders, respHeaders)
if statusCode = 200
    if respHeaders.contains("Content-Type") = false
        throwRuntimeError("couldn't write current chart to file: no response
header Content-Type found")
    elseif regex_search(respHeaders["Content-Type"], "^image/\w+$") == ""
        throwRuntimeError("couldn't write current chart to file: expected
Content-Type image/*, but got \"" + respHeaders["Content-Type"] + "\"")
    end

    writeBytesToFile(bytes, fileName)
end

```

See also

[readBytesFromFile \[SimTalk\]](#)